



Software Factory en la Era de la IA

Casos Aplicacion

Organización	lean-ai
Clasificación	Generado desde sf-kb
Fecha	13 de julio de 2026
Versión	1.0

Transformacion Digital

Transformación Digital

Definición

La transformación digital es el proceso de integrar tecnología digital en todas las áreas de una organización, cambiando fundamentalmente cómo opera y entrega valor a los clientes.

Estrategias de transformación

Estrategia	Descripción	Caso típico
Big Bang	Cambio total en fecha límite	Migración de legado crítico
Evolutiva	Incrementos continuos	Adoptar cloud por equipos
Piloto	Una área pionera valida	Un squad prueba DevOps primero
Fusionada	Adquisición + integración	Comprar startup tecnológica

Fases de una transformación

Fase 1 — Diagnóstico

- Evaluar madurez tecnológica actual
- Mapear procesos y dependencias
- Identificar quick wins y deuda técnica
- Documentar stakeholders y resistencia al cambio
- Ver Modelos de Madurez

Fase 2 — Visión y roadmap

- Definir estado deseado (12-18 meses)
- Establecer métricas de éxito (DORA, NPS, tiempo al mercado)
- Priorizar iniciativas por impacto vs esfuerzo
- Crear coalición de cambio (leadership + tech leads)
- Ver Gestión de Proyectos

Fase 3 — Ejecución incremental

- Lanzar squads piloto con autonomía
- Implementar prácticas core: CI/CD, trunk-based, observabilidad
- Medir y ajustar cada sprint
- Documentar lecciones para escalar

- Ver Checklist Squad

Fase 4 — Escalamiento

- Extender prácticas a todos los squads
- Establecer Platform Engineering IDP
- Gobernanza distribuida con guías, no gates
- Cultura de mejora continua institucionalizada

Factores de éxito

Factor	Impacto	Ver	
Sponsor ejecutivo comprometido	Crítico	[[../11-Cultura-Equipo/README\	Cultura]]
Equipos autónomos y cross-funcionales	Alto	[[../02-Estructura/README\	Estructura]]
Métricas visibles (dashboards)	Alto	[[../06-Metricas/README\	Métricas]]
Inversión en enablement	Alto	[[../04-Practicas/README\	Prácticas]]
Comunicación constante	Medio	[[../10-Gestion-Proyectos/README\	Gestión]]

Casos reales

Caso 1 — Banco regional → Cloud Native

- **Contexto:** Monolito en .NET Framework, deploy trimestral
- **Transformación:** Strangler Fig a microservicios en AKS
- **Duración:** 18 meses
- **Resultado:** Deploy diario, reducción 60% en tiempo de onboarding
- **Lección:** No subestimar la resistencia del equipo de operaciones

Caso 2 — Retail → Omnichannel

- **Contexto:** Tiendas físicas sin integración digital
- **Transformación:** API-first, headless commerce, data platform
- **Duración:** 24 meses
- **Resultado:** +35% revenue digital, inventario unificado
- **Lección:** El cambio cultural fue más difícil que la tecnología

Caso 3 — Manufacturing → IoT + IA

- **Contexto:** Procesos manuales sin datos en tiempo real
- **Transformación:** Sensores IoT, pipeline de datos, dashboards predictivos
- **Duración:** 12 meses piloto, escalamiento en curso
- **Resultado:** -20% downtime, predicción de fallas
- **Lección:** La calidad de datos define el éxito del ML

Anti-patrones en transformación

Ver Anti-patrones para errores frecuentes:

- Transformación sin cambio cultural
- Big Bang sin rollback
- Copiar practices sin contexto
- Ignorar la deuda técnica acumulada

Métricas de progreso

Métrica	Línea base	Objetivo
Deployment Frequency	Mensual	Diaria
Lead Time for Changes	3 meses	< 1 semana
Change Failure Rate	30%	< 5%
MTTR	1 semana	< 1 hora

Ver Métricas DORA para definiciones.

Migracion Cloud

Migración a Cloud: Casos Reales

La estrategia 6 Rs

La estrategia de migración de AWS se resume en 6 opciones para cada workload:

RS	Estrategia	Descripción	Cuándo usar
1	Rehost (Lift & Shift)	Mover sin cambios	Migración urgente, bajo riesgo
2	Replatform (Lift & Reshape)	Optimizaciones menores	Migrar DB a managed services
3	Refactor/Re-architect	Rediseñar para cloud-native	Valor máximo, escala elastic
4	Repurchase	Cambiar de producto	SaaS > self-hosted
5	Retire	Eliminar	Sin uso o redundante
6	Retain	Mantener on-prem	Compliance, dependencias

Patrones de migración

Patrón — Strangler Fig (Migración evolutiva)

Monolito → [Proxy/Facade] → Módulo A (nube) + Módulo B (on-prem)
↓ (fase 2)
Módulo B (nuevamente en nube)

- Ventaja: Migración incremental sin big-bang risk
- Complejidad: Requiere facade estable y trazabilidad
- Ver Monolito a Microservicios

Patrón — Branch by Abstraction

Código → Interface abstractions → Implementación legacy
→ Implementación cloud

- Permite migración sin cambio de comportamiento visible
- Requiere feature flags para control de tráfico
- Ver Feature Flags

Patrón — Database Migration (CDC)

Legacy DB → Debezium/CDC → Kafka → Cloud DB (DynamoDB, CosmosDB)

- Sincronización dual-write durante periodo de transición
- Validación de consistencia antes de corte

Caso real 1 — Fintech: Monolito Java a EKS

Aspecto	Detalle
Stack legacy	Java 8, Oracle DB, WebLogic
Stack target	Java 17, PostgreSQL RDS, EKS
Estrategia	Replatform + Refactor parcial
Duración	12 meses, 4 fases
Resultado	-70% costos infra, deploy continuo

Lecciones aprendidas:

1. El schema de Oracle tenía 800+ stored procedures — automatizar la migración fue esencial
2. La latencia de red al DB requirió connection pooling y cache Redis
3. Los equipos necesitaron 3 meses de capacitación en Kubernetes

Caso real 2 — Gobierno: Reforma legal + Cloud

Aspecto	Detalle
Stack legacy	COBOL mainframe, VSAM files
Stack target	.NET 8, Azure SQL, App Service
Estrategia	Rehost → Refactor incremental
Duración	24 meses (incluyendo capacitación)
Resultado	Cumplimiento normativo, disponibilidad 99.9%

Lecciones aprendidas:

1. La documentación del mainframe era incompleta — interviews con operadores veteranos
2. El testing de regresión automatizado fue el mayor ahorro de tiempo
3. La resistencia del equipo de operaciones requirió programa de upskilling

Checklist pre-migración

- ☐ Inventario completo de workloads y dependencias
- ☐ Evaluación de cada workload para las 6 Rs
- ☐ Análisis de costos (TCO on-prem vs cloud)

- ☐ Definición de orden de migración (low risk first)
- ☐ Plan de rollback para cada fase
- ☐ Capacitación del equipo en servicios target
- ☐ Configuración de landing zone y networking
- ☐ Compliance y requisitos de datos (residencia, encriptación)
- ☐ Monitoring y alertas configurados antes del cutover
- ☐ Documentación de runbooks operacionales

Métricas de éxito

Métrica	Línea base	Target post-migración
Costo mensual infra	\$X	X - 30%
Tiempo de deploy	4 horas	< 30 minutos
Disponibilidad	99.5%	99.95%
MTTR	4 horas	< 30 minutos
Velocidad de onboarding	2 semanas	2 días

Herramientas clave

- AWS Migration Hub, Azure Migrate, Google Migrate
- Debezium (CDC), DMS (Database Migration Service)
- Terraform / Pulumi para IaC IaC
- EKS / AKS / GKE para Kubernetes Kubernetes
- CloudWatch / Azure Monitor / Cloud Monitoring

Adopcion Devops

Adopción de DevOps: Casos Reales

Modelo de madurez DevOps

Nivel	Nombre	Características	Métricas
0	Caótico	Deploy manual, testing manual	DF: < mensual, MTTR: días
1	Inicial	CI básico, version control	DF: semanal, MTTR: horas
2	Repetible	CD pipeline, IaC básico	DF: diario, MTTR: < 1 hora
3	Definido	Infra como código, trunk-based	DF: multiple/día, MTTR: < 30 min
4	Gestionado	Feature flags, canary releases	DF: on-demand, MTTR: < 15 min
5	Optimizado	Auto-remediation, chaos eng	DF: on-demand, MTTR: < 5 min

Ver Modelos de Madurez para CMMI/SPICE.

Fases de adopción

Fase 1 — Fundamentos (Meses 1-3)

Objetivo: Establecer la base técnica y cultural

- ☐ Version control para todo (infra, config, app)
- ☐ CI pipeline básico (build + test automático)
- DevOps Ciclo para framework
 - ☐ Definir Definition of Done que incluya deployabilidad
- DoD como referencia

Métricas iniciales:

- Deployment Frequency: mensual → semanal
- Build Time: medir línea base
- % de código en version control: 100%

Fase 2 — Automatización (Meses 3-6)

Objetivo: Reducir fricción y errores humanos

- ☐ CD pipeline completo (test → staging → prod)
- ☐ Infrastructure as Code para todos los ambientes
- IaC para Terraform/Pulumi
 - ☐ Testing automatizado (unit, integration, e2e)

- TDD como práctica base
 - ☐ Monitoring y alertas básicas
- Observabilidad

Métricas:

- Deployment Frequency: semanal → diario
- Change Failure Rate: medir línea base
- Lead Time: medir línea base

Fase 3 — Observabilidad (Meses 6-9)

Objetivo: Entender el sistema en producción

- ☐ Tracing distribuido (Jaeger, Zipkin)
- ☐ Métricas de negocio + técnica (RED/USE)
- ☐ Runbooks para alertas críticas
- ☐ On-call rotación definida
- SRE para prácticas

Métricas:

- MTTR: horas → < 30 minutos
- % de alertas con runbook: > 80%
- DORA 4 metrics: publicar dashboard

Fase 4 — Excelencia (Meses 9-12)

Objetivo: Mejora continua sistematizada

- ☐ Chaos engineering para resiliencia
- Chaos Engineering
 - ☐ Cost optimization review mensual
- Cost Optimization
 - ☐ Game days y blameless postmortems
 - ☐ Inner source y contribuciones cross-squad
 - ☐ Platform Engineering team
- Plataformas

Métricas:

- DORA 4: Elite level
- SLO compliance: > 99.5%
- Employee NPS: > 50

Caso real — Fintech mediana

Aspecto	Antes	Después (12 meses)
Deploy frequency	Mensual	5x/día
Lead time	6 semanas	2 días
Change failure rate	25%	3%
MTTR	8 horas	20 minutos
Costo infra	\$120K/mes	\$75K/mes

Acciones clave:

1. Crear platform team de 3 personas como enablement
2. Adoptar trunk-based development con feature flags
3. Implementar SLOs y error budgets por servicio
4. Blameless postmortems en cada incidente P1/P2

Anti-patrones en adopción DevOps

Ver Anti-patrones:

- **DevOps = herramientas:** Solo implementar Jenkins sin cambiar cultura
- **Throw it over the wall:** DevOps como equipo dedicado en vez de práctica
- **Tool sprawl:** 5 herramientas de CI diferentes por proyecto
- **Ignoring security:** DevSecOps ignorado hasta el primer incidente

Referencias

- Accelerate (Forsgren, Humble, Kim)
- The DevOps Handbook (Kim, Humble, Debois, Willis)
- State of DevOps Reports (DORA)
- Antología de Pensadores

Ai Adopcion Casos

Adopción de IA en Equipos

Marco de adopción

Niveles de integración IA

Nivel	Descripción	Herramientas	Impacto esperado
0 — Ninguno	Sin herramientas IA	—	Línea base
1 — Asistente	Copilot, code completion	GitHub Copilot, Cursor	+10-25% productividad
2 — Generador	Generación de código y tests	Copilot Chat, Claude	+25-40% productividad
3 — Agente	Tareas autónomas complejas	Devin, OpenCode, Codex CLI	+40-60% productividad
4 — Orquestador	Múltiples agentes coordinados	AutoGen, CrewAI	+60%+ (experimental)

Ver Agentes IA y Arquitectura Agentes.

Programa piloto (6 semanas)

Semana 1-2: Setup y evaluación

- ☐ Seleccionar squad piloto (4-6 devs, mixto seniority)
- ☐ Definir baseline de productividad (velocity, lead time)
- ☐ Instalar herramientas IA y configurar permisos de código
- ☐ Workshop de prompt engineering básico
- Prompt Engineering como referencia
 - ☐ Establecer métricas de éxito: velocity, satisfaction, defect rate
 - ☐ Configurar tracking de uso de herramientas IA

Semana 3-4: Experimentación guiada

- ☐ Tareas asignadas: 50% con IA, 50% sin IA (A/B informal)
- ☐ Diario: compartir wins y friction points (15 min)
- ☐ Documentar casos de uso efectivos vs frustrantes
- ☐ Identificar code patterns donde IA falla consistentemente
- ☐ Medir calidad: code review findings, test coverage delta
- Ver Code Review

Semana 5-6: Evaluación y decisión

- ☐ Analizar métricas: velocity delta, defect rate, satisfaction
- ☐ Presentar resultados a leadership con datos
- ☐ Decisión: expandir, ajustar o abandonar
- ☐ Crear playbook de mejores prácticas del squad piloto
- ☐ Plan de rollout para siguientes squads si proceed

Estrategias de rollout

Estrategia	Ventaja	Riesgo	Cuándo usar
Top-down mandate	Rápido	Resistencia	Cultura ya digital
Champion network	Organic growth	Lento	Equipos diversos
Guerrilla	Bottom-up energy	Inconsistencia	Startups ágiles
Pilot → expand	Validado con datos	Requiere sponsorship	Empresas tradicionales

ROI measurement

Fórmula de ROI

$$ROI = (Beneficio - Costo) / Costo \times 100$$

$$Beneficio = (horas\ ahorradas \times costo/hora) + (reducción\ defects \times costo/defecto)$$

$$Costo = licencias + enablement + tiempo\ de\ aprendizaje$$

Métricas de impacto

Métrica	Cómo medir	Target típico
Velocity change	Story points sprint	+15-30%
Time to first commit (onboarding)	Días	-40%
Code review turnaround	Horas	-30%
Bug escape rate	Bugs por release	-20%
Developer satisfaction	Survey (1-10)	+2 puntos
Time to generate tests	Minutos	-50%

Caso real — Empresa SaaS B2B

Aspecto	Detalle
Tamaño	45 developers, 6 squads

Aspecto	Detalle
Herramienta	GitHub Copilot + Cursor
Piloto	1 squad, 6 semanas
Resultado piloto	+22% velocity, 0 incremento en defectos
Rollout	2 squads/mes durante 3 meses
ROI año 1	340% (ahorro 2,400 dev-hours)

Lecciones:

1. Los seniors adoptan más rápido que juniors
2. La clave es prompt engineering, no la herramienta
3. Code review es crítico: la IA introduce bugs sutiles
4. La satisfacción del equipo subió significativamente

Riesgos y mitigaciones

Riesgo	Mitigación	
Generación de código inseguro	[[../08-Calidad-Seguridad/README\	Security scanning]] obligatorio
Over-reliance en IA	Entrenar en fundamentos primero	
Datos sensibles en prompts	Policies de DLP, self-hosted options	
Código duplicado no detectado	[[../04-Practicas/10-refactoring-continuo\	Refactoring]] regular
Pérdida de conocimiento tácito	Pair programming + documentación	

Ver Limitaciones y Riesgos de IA.

Anti Patrones

Anti-patrones en Software Factories

¿Qué es un anti-patrón?

Un anti-patrón es una práctica que parece útil pero que en realidad contraproducente. Se repite frecuentemente porque es intuitiva o porque funciona a corto plazo.

Anti-patrones organizacionales

Hero Culture

Descripción: Un desarrollador estrella resuelve todo, el equipo depende de una persona.

Señales:

- Hay 1-2 personas que siempre hacen "hotfixes"
- Code review solo pasa por 1-2 senior
- Cuando falta "el experto", nada avanza
- Onboarding lento porque el conocimiento no se documenta

Consecuencias:

- Burnout del hero
- Bottleneck en decisiones técnicas
- Know-how concentrado (single point of failure)
- Baja moral del resto del equipo

Solución:

- Pair programming para distribuir conocimiento
- Pair Programming como práctica estándar
- Documentación de decisiones (ADRs)
- Documentación Técnica
- Rotación de responsabilidades (on-call, code review)

Death March

Descripción: Proyecto con fecha fija, alcance inmutable, y recursos insuficientes.

Señales:

- "Necesitamos que esté listo para [fecha comercial]"
- Scope creep sin reducción de alcance
- Horas extra obligatorias ("temporalmente")
- Cancelación de testing para "cumplir plazos"

Consecuencias:

- Burnout masivo
- Calidad deplorable, bug fixes eternos
- Rotación de talento (el mejor se va primero)
- Deuda técnica masiva

Solución:

- Negociar alcance, no tiempo
- Usar velocity real para forecasting
- Gestión de Proyectos con datos
- Escalar temprano: si falta 30% de capacidad, hay que hablar

Cargo Cult Agile

Descripción: Adoptar ceremonias y artefactos Agile sin el mindset subyacente.

Señales:

- Daily standup de status report (no de sincronización)
- Sprint review es demo para stakeholders, sin feedback real
- Retros sin acción items, siempre "todo bien"
- Story points como métrica de productividad individual
- Ver Procesos para frameworks ágiles

Solución:

- Entrenar en principios, no en ceremonias
- Retros con action items y seguimiento
- Medir outcomes, no output
- Errores Comunes para más detalles

Anti-patrones técnicos

Microservices Premium (antes de tiempo)

Descripción: Adoptar microservicios sin la madurez técnica necesaria.

Señales:

- Equipo de 5 personas con 15 microservicios
- Distributed monolith: todos los servicios acoplados
- Sin observabilidad distribuida
- Deploy de un servicio requiere deploy de 5 más

Solución:

- Comenzar con monolito modular
- Monolito a Microservicios como guía

- Escalar a microservicios solo cuando hay claro bounded context
- Ver Arquitectura Limpia

Resume Driven Development

Descripción: Elegir tecnología por popularidad en CV, no por necesidad.

Señales:

- "Necesitamos Kafka" sin caso de uso de streaming
- Frameworks elegidos por trending en HackerNews
- Reescritura completa para "modernizar" (sin business case)

Solución:

- ADRs para decisiones de tecnología
- Evaluar según criterios: team expertise, ecosystem, operability
- Prefer soluciones boring cuando sea posible

Anti-patrones de proceso

Git Flow Obsoleto

Descripción: Usar Git Flow completo en proyectos que no lo necesitan.

Problema: Feature branches largos, merge conflicts, fear of deploying.

Solución: Trunk-based development con feature flags

- Convenciones Git
- Feature Flags

Test coverage como target

Descripción: Obligar 100% coverage sin medir calidad de tests.

Problema: Tests triviales que no detectan bugs, sensación falsa de seguridad.

Solución: Medir mutation testing, code coverage como guía no como meta

- Calidad y Seguridad

Premature Optimization

Descripción: Optimizar rendimiento antes de tener problemas medibles.

Solución: Measure first, optimize second

- Métricas para datos objetivos

Cómo detectar anti-patrones en tu equipo

Señal de alerta	Anti-patrón probable
Un desarrollador resuelve >50% de bugs críticos	Hero Culture
Sprint constantemente over-committed	Death March
Retros siempre "todo bien"	Cargo Cult Agile
Más de 10 microservicios para 5 devs	Microservices Premium
Nueva tecnología cada trimestre	Resume Driven Development
Deploy de feature toma >1 semana	Git Flow Obsoleto
Test suite lenta y frágil	Test coverage como target

Referencias

- AntiPatterns: Refactoring Software, Architectures (Brown et al.)
- Antología de Pensadores
- Buenas Prácticas

Fallidos Comunes

Errores Comunes y Cómo Evitarlos

Top 10 errores en equipos de software

1. No invertir en onboarding

El error: Asumir que un nuevo dev "se adapta solo" o que 1 semana es suficiente.

Impacto: Meses de bajo rendimiento, código que no sigue convenciones, alta frustración.

Solución:

- ☐ Documentar ambiente de desarrollo (setup guide)
- ☐ Crear "first week" checklist para nuevos
- ☐ Asignar buddy/mentor para primer mes
- ☐ Primer PR simple en el primer día
- Gestión de Proyectos para planificación

2. Skipping testing

El-error: "No tenemos tiempo para tests, luego los escribimos" (nunca se escribe).

Impacto: Regresiones constantes, fear of deploying, velocity cae a largo plazo.

Solución:

- ☐ Test automation como Definition of Done
- ☐ TDD como práctica (no obligatoria pero recomendada)
- ☐ Minimum 80% coverage en código nuevo
- ☐ Mutation testing para validar calidad de tests
- TDD
- Calidad

3. Ignorar la deuda técnica

El error: "Después lo refactorizamos" (nunca se hace).

Impacto: Velocidad decreciente, más bugs, frustración del equipo.

Solución:

- ☐ 20% del capacity para tech debt cada sprint
- ☐ Tech debt backlog visible para product
- ☐ Refactoring oportunistas en feature work
- Technical Debt
- Refactoring

4. Comunicación deficiente entre roles

El error: Devs y product owners no se hablan. Specs ambiguos, expectativas no alineadas.

Impacto: Features mal entendidas, rework, wasted effort.

Solución:

- ☐ Three Amigos sessions antes de cada sprint
- ☐ Acceptance criteria claros en cada story
- ☐ Product y devs en las mismas canales de comunicación
- ☐ Demo frecuente para validar dirección
- Ver Procesos

5. Sin observabilidad en producción

El error: Deploy a producción sin monitoring, logs estructurados, ni alertas.

Impacto: Incidentes no detectados, MTTR alto, debugging por "adivinanza".

Solución:

- ☐ Monitoring desde el día 1 del servicio
- ☐ Logs estructurados (JSON) con correlation IDs
- ☐ Métricas RED (Rate, Errors, Duration)
- ☐ Alertas con runbooks asociados
- Observabilidad
- SRE

6. No automatizar deploys

El-error: Deploy manual que requiere 3 personas y 2 horas.

Impacto: Deploy doloroso = deploy infrecuente = releases grandes = más riesgo.

Solución:

- ☐ CI/CD pipeline completo
- ☐ Deploy automatizado a todos los ambientes
- ☐ Feature flags para desacoplar deploy de release
- CD
- Feature Flags

7. Copy-paste architecture

El error: Copiar soluciones entre proyectos sin considerar contexto.

Impacto: Duplicación, inconsistencias,难以 mantenimiento.

Solución:

- ☐ Shared libraries con ownership claro
- ☐ Module Federation o packages internos

- ☐ Documentar decisiones de arquitectura (ADR)

- Documentación

8. No medir nada

El error: "Confiamos en que va bien" sin datos.

Impacto: Decisiones basadas en opinión, no en evidencia.

Solución:

- ☐ Dashboard de DORA metrics visible para todos
- ☐ SLOs por servicio
- ☐ Regular review de métricas (mensual)

- Métricas

9. Security como afterthought

El error: "Añadimos seguridad al final" (cuando ya hay vulnerabilidades en producción).

Impacto: Brechas, pérdida de datos, multas, daño reputacional.

Solución:

- ☐ SAST/DAST en CI pipeline
- ☐ Secret scanning automatizado
- ☐ Dependency vulnerability scanning
- ☐ Security champions en cada squad

- Seguridad

10. No invertir en la cultura del equipo

El error: Solo enfocarse en velocity, ignorando wellbeing y crecimiento.

Impacto: Burnout, rotación, knowledge loss, baja moral.

Solución:

- ☐ Retros efectivas con action items
- ☐ 1:1s regulares con managers
- ☐ Learning time dedicado
- ☐ Celebration de wins

- Anti-patronos para señales tempranas
- Cultura

Resumen visual

#	Error	Frecuencia	Impacto
1	Onboarding débil	Muy alta	Alto

#	Error	Frecuencia	Impacto
2	Skipping tests	Muy alta	Crítico
3	Deuda técnica ignorada	Alta	Crítico
4	Comunicación mala	Alta	Alto
5	Sin observabilidad	Alta	Crítico
6	Deploy manual	Media	Alto
7	Copy-paste arch	Media	Medio
8	No medir	Media	Alto
9	Security afterthought	Baja	Crítico
10	Ignorar cultura	Baja	Crítico

Buenas Practicas Compendium

Compendio de Buenas Prácticas

Propósito

Resumen ejecutivo de las prácticas más importantes organizadas por dimensión. Cada entrada enlaza con la sección detallada correspondiente.

1. Código y calidad

Práctica	Resumen	Referencia	
Coding Standards	Convenciones consistentes en todo el equipo	[[../04-Practicas/01-coding-standards	Estándares]]
Clean Code	Nombres significativos, funciones pequeñas, DRY	[[../04-Practicas/02-clean-code	Clean Code]]
TDD	Escribir test antes que código	[[../04-Practicas/03-test-driven-development	TDD]]
BDD	Tests legibles por negocio	[[../04-Practicas/04-behavior-driven-development	BDD]]
Code Review	Revisión obligatoria antes de merge	[[../04-Practicas/06-code-review-best-practices	Code Review]]
Refactoring	Mejora continua sin cambiar comportamiento	[[../04-Practicas/10-refactoring-continuo	Refactoring]]

Checklist de código saludable

- ☐ Standards definidos y aplicados con linter
- ☐ Code review para cada PR
- ☐ Test coverage > 80% en código nuevo
- ☐ No code smells críticos (ver Clean Code)
- ☐ Documentation as code (README, ADRs)
- Ver Principios de Ingeniería

2. Procesos y entrega

Práctica	Resumen	Referencia	
CI Pipeline	Build + test automático en cada commit	[[../09-DevOps-Infra/01-devops-ciclo	DevOps]]

Práctica	Resumen	Referencia	
CD Pipeline	Deploy automatizado a producción	[[../09-DevOps-Infra/02-continuous-delivery	CD]]
Trunk-Based Development	Feature branches cortas (< 2 días)	[[../04-Practicas/09-git-conventions	Git]]
Feature Flags	Desacoplar deploy de release	[[../04-Practicas/13-feature-flags	Feature Flags]]
Definition of Done	Criterios claros de completitud	[[../04-Practicas/11-definition-of-done	DoD]]

Checklist de entrega

- ☐ CI corre en < 10 minutos
- ☐ CD puede deployar a producción en < 30 minutos
- ☐ Feature flags para cambios de riesgo alto
- ☐ Rollback automático en caso de error
- ☐ Monitoring post-deploy configurado

3. Infraestructura y operaciones

Práctica	Resumen	Referencia	
Infrastructure as Code	Infra versionada y reproducible	[[../09-DevOps-Infra/03-infrastructure-as-code	IaC]]
Containers	Empaquetado consistente	[[../09-DevOps-Infra/04-containers-docker	Docker]]
Kubernetes	Orchestration para workloads complejos	[[../09-DevOps-Infra/05-kubernetes-orchestration	K8s]]
GitOps	Git como single source of truth para infra	[[../09-DevOps-Infra/07-gitops	GitOps]]
SRE	SLOs, error budgets, incident management	[[../09-DevOps-Infra/08-site-reliability	SRE]]
Cloud Native	Diseño para la nube	[[../09-DevOps-Infra/10-cloud-native	Cloud Native]]
Cost Optimization	FinOps y gobernanza de costos	[[../09-DevOps-Infra/12-cost-optimization	FinOps]]

Checklist de infra

- ☐ Infra declarativa (Terraform/Pulumi)
- ☐ Ambientes reproducibles desde cero
- ☐ Secrets en vault (no en código)

- ☐ Monitoring + alertas para servicios core
- ☐ Incident response process documentado

4. Observabilidad y calidad

Práctica	Resumen	Referencia	
Observabilidad	Logs, métricas, traces	[[../04-Practicas/12-observabilidad-practicas	Observabilidad]]
Chaos Engineering	Probar resiliencia activamente	[[../04-Practicas/14-chaos-engineering	Chaos]]
Performance Testing	Tests de carga en CI	[[../08-Calidad-Seguridad/README	Calidad]]

Checklist de observabilidad

- ☐ Logs estructurados con correlation IDs
- ☐ Métricas RED/USE para servicios críticos
- ☐ Distributed tracing para requests cross-service
- ☐ Dashboards para cada servicio (RED)
- ☐ Alertas con runbooks y on-call rotation

5. Seguridad

Práctica	Resumen	Referencia	
Shift-Left Security	Seguridad desde el diseño	[[../01-Fundamentos/08-shift-left	Shift-Left]]
SAST/DAST en CI	Análisis automático de vulnerabilidades	[[../08-Calidad-Seguridad/README	Seguridad]]
Secret Management	Vault, no hardcoded secrets	[[../09-DevOps-Infra/03-infrastructure-as-code	IaC]]
Dependency Scanning	Actualizar dependencias vulnerables	[[../08-Calidad-Seguridad/README	Seguridad]]

6. Cultura y equipo

Práctica	Resumen	Referencia	
Blameless Postmortems	Aprender de fallos sin señalar culpables	[[../09-DevOps-Infra/09-incident-response	Incident Response]]
Pair Programming	Conocimiento compartido	[[../04-Practicas/05-pair-programming	Pair Programming]]
Learning Time	Tiempo dedicado a crecimiento	[[../11-Cultura-Equipo/README	Cultura]]

Práctica	Resumen	Referencia	
Psychological Safety	Ambiente de seguridad para fallar	[[../11-Cultura-Equipo/README	Cultura]]

7. Gestión de proyectos

Práctica	Resumen	Referencia	
Agile/Scrum/Kanban	Framework de trabajo iterativo	[[../03-Procesos/README	Procesos]]
Estimación basada en datos	Velocidad histórica, no conjetura	[[../06-Metricas/README	Métricas]]
Backlog grooming	Refinamiento continuo de prioridades	[[../10-Gestion-Proyectos/README	Gestión]]

8. IA en desarrollo

Práctica	Resumen	Referencia	
AI Pair Programming	Copilot + developer como duo	[[../07-IA-Software/08-ai-pair-programming	AI Pair]]
AI Code Analysis	Detección de code smells con IA	[[../07-IA-Software/05-ai-code-analysis	AI Analysis]]
AI Test Generation	Generación asistida de tests	[[../07-IA-Software/04-ai-test-generation	AI Tests]]
AI Documentation	Documentación asistida	[[../07-IA-Software/06-ai-documentation	AI Docs]]

Priorización de adopción

Fase	Prácticas	Impacto	Esfuerzo
Semana 1	Coding standards, CI basics, version control	Alto	Bajo
Mes 1	Code review, TDD basics, CD pipeline	Alto	Medio
Mes 3	Observabilidad, feature flags, IaC	Alto	Medio
Mes 6	SRE practices, chaos engineering, FinOps	Medio	Alto
Mes 9	AI tools, advanced patterns	Medio	Medio

Checklist Proyecto

Checklist para Nuevo Proyecto

Instrucciones

Marcar cada ítem al inicio del proyecto. Revisar en la primera retrospectiva y cada trimestre.

1. Arquitectura y diseño

- ☐ Bounded contexts identificados (DDD)
- Arquitectura Limpia como guía
 - ☐ Patrón arquitectónico definido (monolito modular, microservicios, event-driven)
 - ☐ Decisiones documentadas en ADRs
- Documentación Técnica
 - ☐ API contracts definidos (OpenAPI, gRPC, GraphQL)
- API Standards
 - ☐ Data model y estrategia de migración (si aplica)
 - ☐ Non-functional requirements definidos (latencia, throughput, disponibilidad)
 - ☐ Technology stack aprobado y documentado
 - ☐ Dependency graph y versiones fijadas (lock files)
- Ver Software Categoría A vs B

2. Repositorio y código

- ☐ Repository creado con README completo
- ☐ .gitignore configurado (sin secrets)
- ☐ Branching strategy definida (trunk-based recomendado)
- Convenciones Git
 - ☐ Coding standards documentados
- Estándares de Codificación
 - ☐ Linter + formatter configurados (pre-commit hooks)
 - ☐ EditorConfig para consistencia de edición
 - ☐ .env.example con variables requeridas
 - ☐ LICENSE definida
 - ☐ CODEOWNERS configurado

3. CI/CD Pipeline

- ☐ CI pipeline: build + lint + test automatizado
- DevOps Ciclo

- ☐ CD pipeline: deploy automatizado a staging
- CD
 - ☐ Pipeline duration < 10 minutos
 - ☐ Build artifacts versionados y almacenados
 - ☐ Deploy a producción automatizado (feature flags)
- Feature Flags
 - ☐ Rollback automático configurado
 - ☐ Environment secrets en vault (no en pipeline)

4. Testing

- ☐ Test strategy documentada (unit, integration, e2e)
- TDD
 - ☐ Unit tests escritos (coverage $\geq 80\%$ en código nuevo)
 - ☐ Integration tests para puntos de integración
 - ☐ E2E tests para flujos críticos de negocio
 - ☐ Performance tests (si aplica)
 - ☐ Security tests (SAST/DAST en pipeline)
- Calidad y Seguridad
 - ☐ Test data management strategy
 - ☐ Mutation testing habilitado (opcional pero recomendado)

5. Observabilidad

- ☐ Application logs estructurados (JSON)
- Observabilidad
 - ☐ Métricas de negocio y técnica definidas
 - ☐ Distributed tracing configurado
 - ☐ Dashboards iniciales creados
 - ☐ Alertas configuradas para métricas críticas
 - ☐ On-call rotation definida (si aplica)
- SRE

6. Seguridad

- ☐ Threat modeling realizado
- Shift-Left Security
 - ☐ SAST en pipeline de CI
 - ☐ Secret scanning habilitado
 - ☐ Dependency vulnerability scanning
 - ☐ Secrets en vault (no hardcoded)
 - ☐ RBAC / permisos definidos
 - ☐ Data encryption en tránsito y en reposo

- ☐ Compliance requirements documentados (GDPR, SOC2, etc.)

7. Documentación

- ☐ README: setup instructions, architecture overview
- ☐ API documentation (auto-generada desde specs)
- ☐ ADRs para decisiones técnicas clave
- Documentación
 - ☐ Runbook para operaciones críticas
 - ☐ CONTRIBUTING.md para contributors externos
 - ☐ Architecture diagram (C4 o similar)

8. Gestión de proyecto

- ☐ Backlog inicial priorizado
- Gestión de Proyectos
 - ☐ Sprint cadence definido
 - ☐ Definition of Done acordada
- DoD
 - ☐ Stakeholders identificados
 - ☐ Communication plan (slack channels, meetings)
 - ☐ Risk register inicial
 - ☐ Success metrics definidos

9. Infraestructura

- ☐ IaC para todos los ambientes
- IaC
 - ☐ Landing zone configurada
 - ☐ Networking (VPC, subnets, security groups)
 - ☐ DNS y certificates
 - ☐ Backup strategy definida
 - ☐ Disaster recovery plan documentado
 - ☐ Cost estimation y budget aprobado
- Cost Optimization

10. Equipo y proceso

- ☐ Squad configurado
- Checklist Squad
 - ☐ Roles y responsabilidades claros (DACI o RACI)
 - ☐ Ceremony calendar definido
 - ☐ Tools onboarding completado
 - ☐ Incident response process documentado

- Incident Response
 - ☐ Escalation path definido
 - ☐ Retrospective cadence establecido

Resumen de priorización

Prioridad	Secciones	Deadline
P0 — Blocking	Arquitectura, Repo, CI/CD, Security	Día 1
P1 — Required	Testing, Observabilidad, Docs	Semana 1
P2 — Important	Gestión, Infra, Equipo	Mes 1
P3 — Nice to have	Chaos engineering, cost optimization	Trimestre 1

Checklist Squad

Checklist para Nuevo Squad

Instrucciones

Completar al formar un nuevo squad. Revisar en la primera retrospective (2 semanas) y mensualmente.

1. Composición del equipo

- ☐ Tamaño adecuado: 5-9 personas (2 pizza teams)
- ☐ Roles cubiertos: dev, QA/SDET, designer, product owner
- ☐ Mix de seniority equilibrado (no todos seniors ni todos juniors)
- ☐ Tech lead asignado con experiencia demostrada
- ☐ Scrum master o facilitador designado
- ☐ Area de dominio delimitada (bounded context)
- Ley de Conway para align con arquitectura
- Estructura para frameworks organizacionales

2. Herramientas de trabajo

- ☐ Código fuente: repositorio con acceso para todos
- ☐ CI/CD: pipeline configurado y funcionando
- ☐ Comunicación: slack/teams channel dedicado
- ☐ Project management: Jira/Linear/Linear board
- ☐ Documentación: wiki/confluence/notion con espacio propio
- ☐ Design: Figma/shared design system access
- ☐ Monitoring: acceso a dashboards de servicios propios
- ☐ Incident management: PagerDuty/Opsgenie configurado

3. Procesos

Ceremony calendar

Ceremony	Frecuencia	Duración	Participantes
Daily standup	Diaria	15 min	Todo el squad
Sprint planning	Cada 2 semanas	2-4 horas	Todo el squad
Backlog refinement	Semanal	1-2 horas	PO + devs
Sprint review/Demo	Cada 2 semanas	1 hora	Squad + stakeholders

Ceremony	Frecuencia	Duración	Participantes
Retrospective	Cada 2 semanas	1-1.5 horas	Solo squad
1:1s (manager)	Semanal	30 min	Individual

Definition of Done

- ☐ DoD acordada por todo el squad
- DoD como plantilla
 - ☐ Incluye: code review, tests, documentation, security scan
 - ☐ Revisable y actualizable en cada retro

Code practices

- ☐ Coding standards acordados
- Estándares
 - ☐ Branching strategy definida (trunk-based recomendado)
- Git Conventions
 - ☐ Code review mandatory para todos los PRs
- Code Review
 - ☐ Pair programming sessions programadas
- Pair Programming

4. Acuerdos de trabajo

- ☐ Horarios core (overlap mínimo de 4 horas)
- ☐ Política de remote/hybrid definida
- ☐ Expectativas de respuesta (no async para urgencias)
- ☐ No-meeting blocks respetados
- ☐ Learning time semanal (ej: Friday afternoon)
- ☐ Onboarding plan para nuevos miembros
- Cultura para acuerdos culturales

5. Métricas del squad

- ☐ Velocity baseline medida (3 sprints mínimos)
- ☐ DORA metrics visibles (deploy freq, lead time, MTTR, CFR)
- Métricas
 - ☐ Sprint burndown/burnup visible
 - ☐ Team health check mensual (anonymous survey)
 - ☐ Quality metrics: test coverage, bug escape rate
 - ☐ Satisfaction score (NPS interno)

6. Seguridad y compliance

- ☐ Accesos a producción solo para who-need
- ☐ Least privilege principle para tools
- ☐ Secret management process definido
- ☐ Incident response plan documentado
- Seguridad
- Incident Response

7. Relación con otros squads

- ☐ Inter-squad dependencies mapeadas
- ☐ API contracts con otros squads definidos
- ☐ Shared libraries ownership claro
- ☐ Escalation path inter-squad definido
- ☐ Regular sync con squads dependientes (si aplica)

8. Primer sprint

- ☐ Sprint goal claro y comunicado
- ☐ Backlog refinado para al menos 2 sprints
- ☐ Environment de desarrollo funcionando
- ☐ Primer PR merged por cada miembro nuevo
- ☐ Primer deploy exitoso a staging
- ☐ Celebración del primer deploy (no trivializar)

Checklist de madurez (mes 3)

- ☐ Velocity estable (varianza < 20%)
- ☐ CI/CD maduro (< 10 min pipeline)
- ☐ Test coverage > 80%
- ☐ Retros con action items seguidos
- ☐ Zero P1 incidents sin resolver > 24h
- ☐ Team satisfaction > 7/10
- ☐ Tech debt backlog visible y priorizado

Playbook Transformacion

Playbook de Transformación

Visión general

Guía paso a paso para transformar una organización de software tradicional a una Software Factory moderna, product-oriented y data-driven.

Fase 0 — Preparación (Semanas 1-4)

Objetivo

Generar urgencia, alinear leadership y construir la coalición de cambio.

Acciones

Semana 1: Diagnóstico

- ☐ Medir DORA metrics actuales
- Métricas DORA
 - ☐ Evaluar madurez con CMMI/SPICE
- Modelos de Madurez
 - ☐ Entrevistar 10-15 personas clave (devs, leads, product)
 - ☐ Mapear pain points y quick wins
 - ☐ Documentar estado actual: tech stack, procesos, herramientas

Semana 2: Visión

- ☐ Definir "estado deseado" a 18 meses
- ☐ Traducir visión a métricas medibles
- ☐ Crear narrativa de transformación (storytelling)
- ☐ Identificar 3-5 quick wins de alto impacto, bajo esfuerzo

Semana 3: Coalición

- ☐ Formar transformation team (3-5 personas, dedicadas 50%+)
- ☐ Sponsor ejecutivo comprometido (C-level o VP Eng)
- ☐ Tech leads como champions por área
- ☐ Communication plan para toda la organización

Semana 4: Plan

- ☐ Roadmap de 12 meses con fases
- ☐ Presupuesto estimado (personas, herramientas, enablement)
- ☐ Riesgos y mitigaciones documentados
- ☐ Go/no-go con leadership

Salida esperada

- Documento de diagnóstico + visión
 - Aprobación de presupuesto y equipo dedicado
 - Coalición de cambio constituida
-

Fase 1 — Fundamentos (Meses 1-3)

Objetivo

Establecer la base técnica y cultural que permita la aceleración.

Acciones por área

Procesos

- ☐ Adoptar Scrum o Kanban como framework base
- Procesos
 - ☐ Definir Definition of Done para toda la organización
- DoD
 - ☐ Implementar retros efectivas (con action items)
 - ☐ Establecer cadencia de demos a stakeholders

Técnico

- ☐ Migrar TODO a version control
- ☐ CI pipeline para todos los proyectos
- DevOps
 - ☐ Coding standards + linters
- Estándares
 - ☐ Code review obligatorio
- Code Review

Herramientas

- ☐ Estandarizar stack de herramientas
- Herramientas
 - ☐ Setup de development environments reproducibles
 - ☐ Acceso a dashboards de métricas para todos

Cultura

- ☐ Workshop de mentalidad product-oriented
- Producto vs Proyecto
 - ☐ Blameless postmortems como práctica
 - ☐ Learning time institucionalizado

Métricas de esta fase

Métrica	Target
% proyectos con CI	> 80%
Deploy frequency	Semanal
Code review participation	> 90% PRs

Fase 2 — Automatización (Meses 3-6)

Objetivo

Eliminar manual work y reducir time-to-market.

Acciones

- ☐ CD pipeline completo (auto-deploy a staging)
- CD
 - ☐ Infrastructure as Code para nuevos ambientes
- IaC
 - ☐ Testing automatizado: unit, integration, e2e
- TDD
 - ☐ Feature flags para releases controlados
- Feature Flags
 - ☐ Monitoring y alertas para servicios core
- Observabilidad
 - ☐ Trunk-based development como default
- Git

Métricas de esta fase

Métrica	Target
Deploy frequency	Diaria
Lead time for changes	< 1 semana
Change failure rate	< 15%

Fase 3 — Observabilidad y SRE (Meses 6-9)

Objetivo

Saber qué pasa en producción y responder rápidamente.

Acciones

- ☐ Distributed tracing implementado
- ☐ SLOs definidos por servicio
- SRE
 - ☐ On-call rotation operativa
- Incident Response
 - ☐ Runbooks para top 10 incidentes
 - ☐ Blameless postmortems en cada P1/P2
 - ☐ Cost optimization reviews mensuales
- FinOps

Métricas de esta fase

Métrica	Target
MTTR	< 30 minutos
SLO compliance	> 99.5%
Alertas con runbook	> 80%

Fase 4 — Excelencia y escalamiento (Meses 9-12+)

Objetivo

Institucionalizar la mejora continua y escalar practices.

Acciones

- ☐ Platform Engineering team
- IDP
 - ☐ Chaos engineering para servicios críticos
- Chaos
 - ☐ Inner source para compartir código entre squads
 - ☐ AI tools para development
- IA y Software
 - ☐ Community of practices (guilds)
 - ☐ Regular architecture reviews
- Arquitectura Limpia

Métricas de esta fase (Elite DORA)

Métrica	Target
Deploy frequency	On-demand (múltiples/día)
Lead time for changes	< 1 hora
Change failure rate	< 5%
MTTR	< 1 hora

Factores críticos de éxito

Factor	Por qué importa
Sponsor ejecutivo	Sin soporte de arriba, la transformación muere
Datos, no opiniones	Medir baseline y progreso con métricas reales
Quick wins tempranos	Generan momentum y confianza
Enablement > mandates	Enseñar, no obligar
Paciencia	Transformaciones toman 12-24 meses reales
Celebrar hits	Reconocer progreso mantiene energía

Errores fatales a evitar

Ver Anti-patrones y Errores Comunes:

- Transformar sin cambiar cultura
- Copiar practices de otra empresa sin contexto
- Ignorar la resistencia al cambio
- No medir baseline antes de empezar
- Prometer resultados en 3 meses (son 12-18)

Antología Pensadores

Antología de Pensadores Clave

Propósito

Referencia rápida de los autores que han definido la ingeniería de software moderna. Cada entrada incluye sus contribuciones principales y dónde aplicar sus ideas.

Robert C. Martin (Uncle Bob)

Área: Clean Code, Arquitectura Limpia, ética profesional

Obras clave:

- *Clean Code* (2008) — Principios de código legible
- *Clean Architecture* (2017) — Arquitectura en capas con regla de dependencia
- *The Clean Coder* (2011) — Ética profesional del desarrollador

Ideas principales:

- SOLID principles aplicados a diseño de clases y arquitectura
 - Código limpio como responsabilidad profesional
 - La arquitectura debe ser independiente del framework
 - Ver Principios SOLID
 - Ver Arquitectura Limpia
 - Ver Clean Code
-

Martin Fowler

Área: Patrones de refactorización, arquitectura de software, continuous integration

Obras clave:

- *Refactoring* (1999, 2018 2nd ed.) — Catálogo de refactorizaciones
- *Patterns of Enterprise Application Architecture* (2002) — Patrones enterprise
- *Continuous Integration* (2006) — Práctica de CI

Ideas principales:

- Refactoring como disciplina continua, no actividad puntual
- Patrones como soluciones probadas a problemas recurrentes
- La importancia de la integración continua
- Ver Refactoring
- Ver CI
- Ver Monolito a Microservicios

Jez Humble

Área: Continuous Delivery, DevOps, transformación organizacional

Obras clave:

- *Continuous Delivery* (2010, con David Farley) — CD pipeline y practices
- *Accelerate* (2018, con Forsgren y Kim) — DORA metrics
- *The DevOps Handbook* (2016, co-author)

Ideas principales:

- Deployment pipeline como first-class citizen
 - Trunk-based development sobre feature branches
 - Las 4 claves de la transformación: generación, investigación, simplificación, liderazgo
 - Ver CD
 - Ver DORA Metrics
-

Gene Kim

Área: DevOps, Three Ways, transformación a escala

Obras clave:

- *The Phoenix Project* (2013) — Novela sobre DevOps
- *The DevOps Handbook* (2016, co-author)
- *Accelerate* (2018, co-author)
- *The Unicorn Project* (2019) — Perspectiva del developer

Ideas principales:

- **The Three Ways:** Flow, Feedback, Continual Learning
 - Theory of Constraints aplicada a software delivery
 - Transformación cultural como prerequisite
 - Ver DevOps
 - Ver DevOps Filosofía
-

Kent Beck

Área: Extreme Programming, TDD, diseño ágil

Obras clave:

- *Extreme Programming Explained* (1999, 2004) — XP practices
- *Test Driven Development: By Example* (2002) — TDD paso a paso
- *Tidy First?* (2023) — Prioridades de refactorización

Ideas principales:

- TDD como disciplina de diseño, no solo testing
 - Simple Design: lo que corre, lo que pasa tests, sin duplicación, expresivo
 - El costo del cambio no es constante
 - Ver TDD
 - Ver Manifiesto Ágil
-

Dan North

Área: BDD, DevOps coaching, simplificación de ideas complejas

Obras clave:

- Artículos sobre BDD (2006+)
- Introducción del concepto "behavior" en testing
- Charlas sobre coaching y simplificación

Ideas principales:

- BDD es sobre conversación, no sobre herramientas
 - "Given-When-Then" como formato de especificación
 - DevOps como resultado de colaboración, no de team dedicado
 - Ver BDD
-

Sam Newman

Área: Microservicios, arquitectura distribuida

Obras clave:

- *Building Microservices* (2015, 2021 2nd ed.) — Guía definitiva de microservicios
- *Monolith to Microservices* (2019) — Patrones de migración

Ideas principales:

- Microservicios como opción arquitectónica, no como meta
 - Bounded contexts como base para servicios
 - La importancia de la observabilidad en sistemas distribuidos
 - Ver Monolito a Microservicios
 - Ver Service Mesh
-

Nicole Forsgren

Área: DORA metrics, investigación empírica en ingeniería

Obras clave:

- *Accelerate* (2018, con Humble y Kim) — State of DevOps
- State of DevOps Reports (2014-2019)

Ideas principales:

- Las 4 métricas de DORA como predictor de performance organizacional
- La cultura informa las prácticas, que habilitan la tecnología
- AI/ML como factor de aceleración
- Ver DORA Metrics
- Ver IA y Software

Thoughtworks Technology Radar

Área: Tendencias tecnológicas, evaluación de herramientas y prácticas

Formato: Publicación semestral con cuadrantes: Techniques, Tools, Languages & Frameworks, Platforms

Ideas principales:

- Clasificar tecnología en: Adopt, Trial, Assess, Hold
- Evaluar tecnología según contexto, no hype
- Mantenerse actualizado sin ser early adopter compulsivo
- Ver Herramientas

Resumen de influencias

Autor	Impacto en esta KB
Robert C. Martin	Clean Code, SOLID, Arquitectura Limpia
Martin Fowler	Refactoring, Patrones, CI
Jez Humble	CD, DORA metrics, Trunk-based
Gene Kim	DevOps, Three Ways, transformación
Kent Beck	TDD, XP, Simple Design
Dan North	BDD, coaching
Sam Newman	Microservicios, migración
Nicole Forsgren	Métricas, investigación empírica

Lecturas recomendadas por nivel

Principiante

1. *The Phoenix Project* — Gene Kim
2. *Clean Code* — Robert C. Martin
3. *Accelerate* — Forsgren, Humble, Kim

Intermedio

1. *Continuous Delivery* — Humble & Farley
2. *Refactoring* — Martin Fowler
3. *Building Microservices* — Sam Newman

Avanzado

1. *Clean Architecture* — Robert C. Martin
2. *The DevOps Handbook* — Kim, Humble, Debois, Willis
3. *Monolith to Microservices* — Sam Newman

Glosario Avanzado

Glosario Avanzado y Siglas

Propósito

Referencia rápida de términos técnicos, siglas y conceptos utilizados en toda la knowledge base. Ordenados alfabéticamente para búsqueda rápida.

A

Término	Definición	Sección	
ADR (Architecture Decision Record)	Documento que registra una decisión de arquitectura significativa	[[../04-Practicas/07-documentacion-tecnica\	Documentación]]
API (Application Programming Interface)	Contrato de comunicación entre sistemas	[[../04-Practicas/08-api-design-standards\	API Standards]]
BDD (Behavior-Driven Development)	Testing centrado en comportamiento con lenguaje natural	[[../04-Practicas/04-behavior-driven-development\	BDD]]
BLD (Build, Link, Deploy)	Fases del pipeline de CI/CD	[[../09-DevOps-Infra/01-devops-ciclo\	DevOps]]

C

Término	Definición	Sección	
CALMS	Culture, Automation, Lean, Measurement, Sharing — pilares DevOps	[[../01-Fundamentos/07-devops-filosofia\	DevOps Filosofía]]
CANARY	Estrategia de release: deploy a porcentaje pequeño de usuarios	[[../04-Practicas/13-feature-flags\	Feature Flags]]
CDC (Change Data Capture)	Patrón de sincronización de datos entre sistemas	[[../12-Casos-Aplicacion/02-migracion-cloud\	Migración Cloud]]
CFR (Change Failure Rate)	% de deploys que causan fallo en producción	[[../06-Metricas/README\	Métricas]]
C4 Model	Modelo de documentación de arquitectura: Context, Container, Component, Code	[[../04-Practicas/07-documentacion-tecnica\	Documentación]]

Término	Definición	Sección	
CI (Continuous Integration)	Práctica de integrar código frecuentemente con build automatizado	[[../09-DevOps-Infra/01-devops-ciclo\	DevOps]]
CMMI (Capability Maturity Model Integration)	Modelo de madurez de procesos	[[../01-Fundamentos/05-modelos-madurez\	Modelos de Madurez]]

D

Término	Definición	Sección	
DAST (Dynamic Application Security Testing)	Testing de seguridad en aplicación corriendo	[[../08-Calidad-Seguridad/README\	Seguridad]]
DDD (Domain-Driven Design)	Diseño centrado en el dominio del negocio	[[../01-Fundamentos/11-arquitectura-limpia\	Arquitectura Limpia]]
DoD (Definition of Done)	Criterios que deben cumplirse para considerar un item completo	[[../04-Practicas/11-definition-of-done\	DoD]]
DORA (DevOps Research & Assessment)	Métricas de performance de software delivery	[[../06-Metricas/README\	Métricas]]
DRY (Don't Repeat Yourself)	Principio de no duplicación de conocimiento	[[../04-Practicas/02-clean-code\	Clean Code]]

F

Término	Definición	Sección	
FinOps	Práctica de gestión financiera de cloud computing	[[../09-DevOps-Infra/12-cost-optimization\	Cost Optimization]]

G

Término	Definición	Sección	
GitOps	Modelo de gestión de infraestructura basado en Git	[[../09-DevOps-Infra/07-gitops\	GitOps]]

I

Término	Definición	Sección	
IaC (Infrastructure as Code)	Gestión de infra declarativa con código versionado	[[../09-DevOps-Infra/03-infrastructure-as-code\	IaC]]

Término	Definición	Sección	
IDP (Internal Developer Platform)	Plataforma interna para developer experience	[[../01-Fundamentos/13-plataformas-ingenieria\	Plataformas]]
IoT (Internet of Things)	Red de dispositivos conectados	[[../12-Casos-Aplicacion/01-transformacion-digital\	Transformación]]

K

Término	Definición	Sección	
KPI (Key Performance Indicator)	Indicador clave de rendimiento	[[../06-Metricas/README\	Métricas]]

L

Término	Definición	Sección	
LoD (Law of Demeter)	Principio de mínima ignorancia: solo hablar con amigos	[[../01-Fundamentos/04-principios-ingenieria-software\	Principios]]
LTFC (Lead Time for Changes)	Tiempo desde commit hasta producción	[[../06-Metricas/README\	Métricas]]

M

Término	Definición	Sección	
MCP (Model Context Protocol)	Protocolo para conectar LLMs con herramientas externas	[[../07-IA-Software/19-mcp-protocol\	MCP Protocol]]
ML (Machine Learning)	Aprendizaje automático	[[../07-IA-Software/README\	IA y Software]]
MTTR (Mean Time To Recovery)	Tiempo medio de recuperación ante incidentes	[[../06-Metricas/README\	Métricas]]

O

Término	Definición	Sección	
OpenAPI	Especificación para describir APIs REST	[[../04-Practicas/08-api-design-standards\	API Standards]]

P

Término	Definición	Sección	
PR (Pull Request)	Mecanismo de revisión de código antes de merge	[[../04-Practicas/06-code-review-best-practices\	Code Review]]

Término	Definición	Sección	
PdC (Plataforma de Componentes)	Repositorio compartido de componentes UI	[[../02-Estructura/README\	Estructura]]

R

Término	Definición	Sección	
RED (Rate, Errors, Duration)	Framework de métricas para servicios	[[../04-Practicas/12-observabilidad-practicas\	Observabilidad]]
RFC (Request for Comments)	Documento de propuesta técnica para discusión	[[../04-Practicas/07-documentacion-tecnica\	Documentación]]
ROI (Return on Investment)	Retorno de inversión	[[../12-Casos-Aplicacion/04-ai-adopcion-casos\	Adopción IA]]
Rs (6)	Framework de migración cloud: Rehost, Replatform, Refactor, Repurchase, Retire, Retain	[[../12-Casos-Aplicacion/02-migracion-cloud\	Migración Cloud]]

S

Término	Definición	Sección	
SAST (Static Application Security Testing)	Análisis de seguridad estático del código	[[../08-Calidad-Seguridad/README\	Seguridad]]
SLO (Service Level Objective)	Objetivo de nivel de servicio medible	[[../09-DevOps-Infra/08-site-reliability\	SRE]]
SLA (Service Level Agreement)	Acuerdo de nivel de servicio con clientes	[[../09-DevOps-Infra/08-site-reliability\	SRE]]
SoC (Separation of Concerns)	Principio de separación de responsabilidades	[[../01-Fundamentos/04-principios-ingenieria-software\	Principios]]
SRE (Site Reliability Engineering)	Práctica de confiabilidad de sitios web	[[../09-DevOps-Infra/08-site-reliability\	SRE]]

T

Término	Definición	Sección	
TDD (Test-Driven Development)	Desarrollo guiado por tests: rojo-verde-refactorizar	[[../04-Practicas/03-test-driven-development\	TDD]]
TF (Trunk-Based Development)	Estrategia de branching con commits directos a main	[[../04-Practicas/09-git-conventions\	Git]]

U

Término	Definición	Sección	
USE (Utilization, Saturation, Errors)	Framework de métricas para recursos	[[../04-Practicas/12-observabilidad-practicas\	Observabilidad]]

W

Término	Definición	Sección	
WIP (Work In Progress)	Trabajo en progreso, limitar para mejorar flow	[[../03-Procesos/README\	Procesos]]

Siglas frecuentes (referencia rápida)

Sigla	Expansión	Contexto
AGILE	Metodología ágil de desarrollo	Procesos
AWS	Amazon Web Services	Cloud
AKS	Azure Kubernetes Service	Cloud
CI/CD	Continuous Integration / Continuous Delivery	DevOps
DDD	Domain-Driven Design	Arquitectura
EKS	Elastic Kubernetes Service	Cloud
GKE	Google Kubernetes Engine	Cloud
IDP	Internal Developer Platform	Platform Eng
K8s	Kubernetes	Containers
LLM	Large Language Model	IA
MCP	Model Context Protocol	IA Tools
RBAC	Role-Based Access Control	Seguridad
SOC2	Service Organization Control 2	Compliance

Ver también

- Antología de Pensadores para autores y sus obras
- Buenas Prácticas para resumen ejecutivo
- Transformación Digital para contexto estratégico

README

Casos y Aplicación

Propósito

Esta sección consolida teoría en práctica concreta. Cada archivo ofrece casos reales, checklists aplicables, playbooks ejecutables y lecciones aprendidas de transformaciones, adopciones y errores comunes en Software Factories.

Mapa de la sección

#	Archivo	Tema	Nivel	
01	[[../12-Casos-Aplicacion/01-transformacion-digital	Transformación Digital]]	Estrategias, fases y factores de éxito	Avanzado
02	[[../12-Casos-Aplicacion/02-migracion-cloud	Migración a Cloud]]	6 Rs, patrones y lecciones	Avanzado
03	[[../12-Casos-Aplicacion/03-adopcion-devops	Adopción de DevOps]]	Modelo de madurez y métricas	Intermedio
04	[[../12-Casos-Aplicacion/04-ai-adopcion-casos	Adopción de IA]]	Pilotos, rollout y ROI	Intermedio
05	[[../12-Casos-Aplicacion/05-anti-patrones	Anti-patrones]]	Hero culture, death marches y más	Avanzado
06	[[../12-Casos-Aplicacion/06-fallidos-comunes	Errores Comunes]]	Top 10 errores y cómo evitarlos	Intermedio
07	[[../12-Casos-Aplicacion/07-buenas-practic-as-compendium	Buenas Prácticas]]	Resumen de prácticas clave	Principiante
08	[[../12-Casos-Aplicacion/08-checklist-proyecto	Checklist Proyecto]]	Nuevo proyecto completo	Intermedio
09	[[../12-Casos-Aplicacion/09-checklist-squad	Checklist Squad]]	Setup de nuevo equipo	Intermedio
10	[[../12-Casos-Aplicacion/10-playbook-transformacion	Playbook Transformación]]	Guía paso a paso	Avanzado
11	[[../12-Casos-Aplicacion/11-antologia-pensadores	Antología de Pensadores]]	Autores clave de la industria	Principiante
12	[[../12-Casos-Aplicacion/12-glosario-avanzado	Glosario Avanzado]]	Términos y siglas extendidas	Principiante

Cómo usar esta sección

1. **Nuevo proyecto:** Comienza con el Checklist de Proyecto y el Checklist de Squad.
2. **Transformación:** Usa el Playbook y revisa los casos de transformación.
3. **Evitar errores:** Consulta Errores Comunes y Anti-patrones.
4. **Referencia rápida:** Consulta el Glosario y la Antología.

Secciones relacionadas

- Fundamentos
- Procesos
- Prácticas
- IA y Software
- DevOps e Infra
- Gestión de Proyectos
- Cultura y Equipo